

MAILYTCUIDA — PROPUESTA ARQUITECTÓNICA COMPLETA

1. ESTADO ACTUAL DE LA APP

Tecnología actual

Plataforma: Android nativo (Kotlin + Jetpack Compose)

DB local: Room (SQLite) versión 7

DB remota: Cloud Firestore (NoSQL)

Auth: Firebase Authentication + Google Sign-In

Crash reporting: Firebase Crashlytics (instalado, sin init manual)

Background: WorkManager + AlarmManager

Storage: Firebase Storage

Módulos existentes (10 módulos, 29 pantallas)

#	Módulo	Pantallas
1	Autenticación	Initial, Login, Signup
2	Home	HomeScreen
3	Medicamentos (CORE)	PatternScreen, MealSchedule, ControlPanel, History, ConfirmationDialog
4	Signos Vitales	Enfermeria, SignosVitales, RegistroIndividual, 13x VitalGrafica
5	Recetas & Lab	RecetasScreen, LaboratorioScreen, PdfScreen
6	Calendario	CalendarioScreen
7	Encuestas	SatisfactionSurvey, AdminSurvey, SurveyCode, CodeManagement, AdminCharts
8	Perfil	ProfileScreen
9	Administración	UserManagement, SendDocument, DocumentHistory
10	Especialistas/Shop	SpecialistScreen, ShopScreen, IAScreen
0	/IA	

Servicios externos actuales

Firebase Auth → reemplazar con Clerk

Firestore → reemplazar con PostgreSQL via Django

Firebase Crashlytics → cambiar con Sentry

Firebase Storage → migrar a S3/Cloudflare R2

Google Sign-In → integrar via Clerk (ya tiene soporte nativo)

2. PROPUESTA DE ARQUITECTURA COMPLETA

Decisión de stack — Justificación

El usuario propone Python/Django. El análisis del dominio confirma que es la elección correcta por:

Datos médicos requieren integridad relacional (FK entre paciente-doctor-medicamento-vital)

Django ORM + PostgreSQL maneja perfectamente las consultas analíticas del dashboard

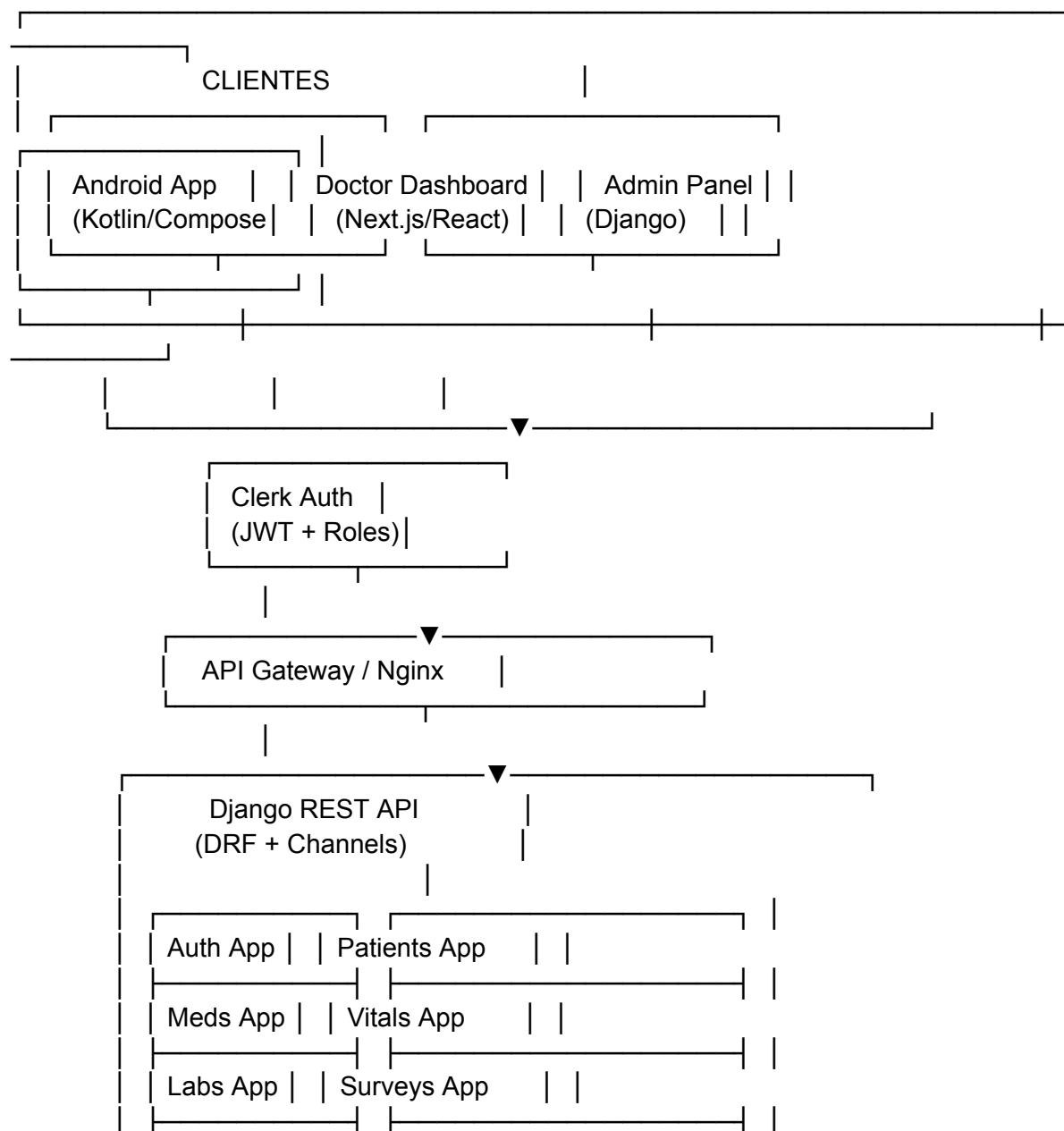
Django Admin es una base sólida para el panel del doctor

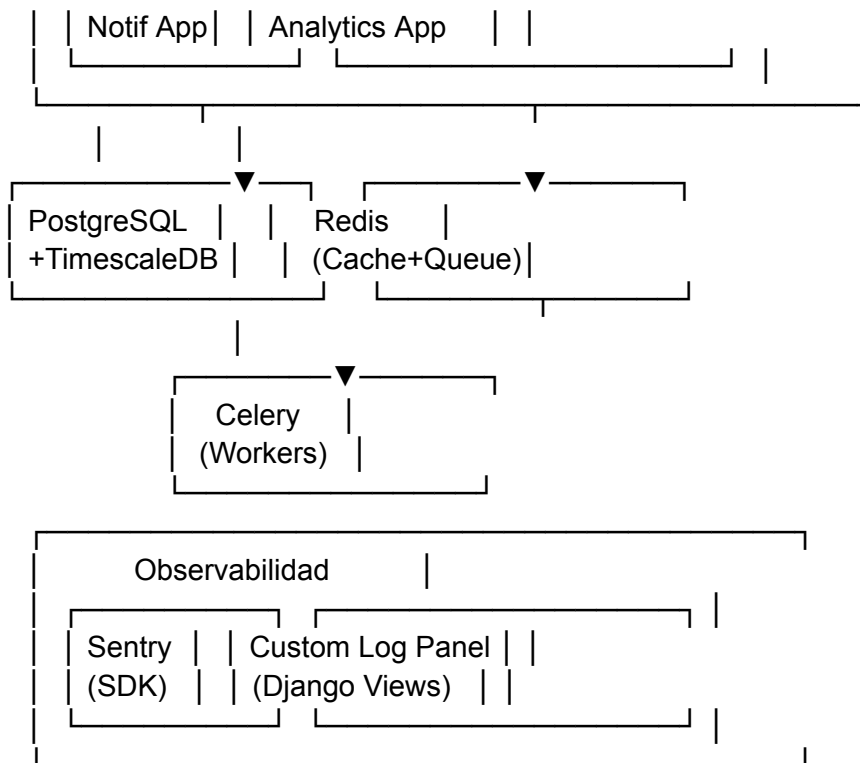
DRF (Django REST Framework) provee API robusta para la app Android

Celery integra nativamente con Django para notificaciones background

TimescaleDB (extensión Postgres) es ideal para series de tiempo de vitales

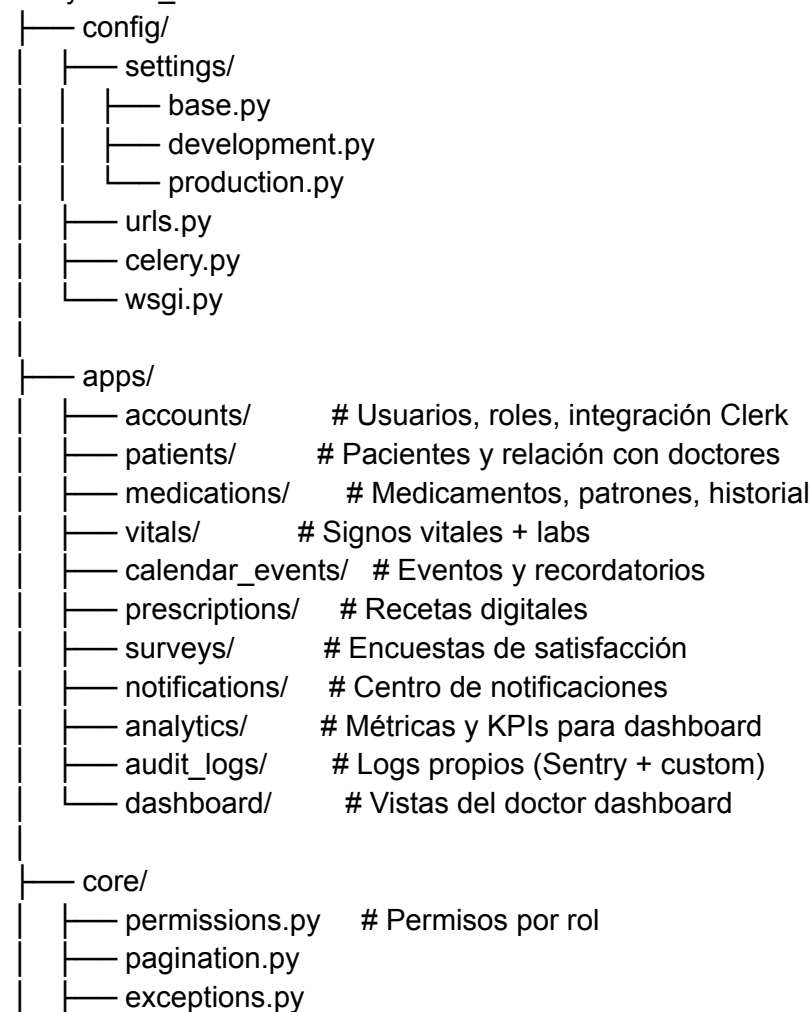
Diagrama de arquitectura general





3. ESTRUCTURA DJANGO — APPS Y MÓDULOS

mailytcuida_backend/



```

|   └─ utils.py
|
└─ requirements/
    ├── base.txt
    ├── development.txt
    └─ production.txt

```

4. ESQUEMAS DE BASE DE DATOS (PostgreSQL)

-- Usuarios (sincronizado con Clerk)

```

CREATE TABLE accounts_user (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  clerk_id    VARCHAR(255) UNIQUE NOT NULL, -- ID de Clerk
  email       VARCHAR(254) UNIQUE NOT NULL,
  phone       VARCHAR(20),
  role        VARCHAR(20) NOT NULL,         -- PATIENT | DOCTOR | ADMIN
  is_active   BOOLEAN DEFAULT TRUE,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  updated_at  TIMESTAMPTZ DEFAULT NOW()
);

```

-- Perfil del paciente

```

CREATE TABLE accounts_patient_profile (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID UNIQUE REFERENCES accounts_user(id) ON DELETE CASCADE,
  first_name  VARCHAR(100) NOT NULL,
  last_name   VARCHAR(100) NOT NULL,
  birth_date  DATE,
  sex         VARCHAR(10),                -- M | F | OTHER
  blood_type  VARCHAR(5),
  allergies   TEXT[],                     -- Array de alergias
  chronic_conditions TEXT[],
  emergency_contact_name VARCHAR(200),
  emergency_contact_phone VARCHAR(20),
  photo_url   VARCHAR(500),
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  updated_at  TIMESTAMPTZ DEFAULT NOW()
);

```

-- Perfil del doctor

```

CREATE TABLE accounts_doctor_profile (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID UNIQUE REFERENCES accounts_user(id) ON DELETE CASCADE,
  first_name  VARCHAR(100) NOT NULL,
  last_name   VARCHAR(100) NOT NULL,
  license_number VARCHAR(100) UNIQUE NOT NULL,
  specialty   VARCHAR(100),
  hospital    VARCHAR(200),
  photo_url   VARCHAR(500),

```

```

        created_at    TIMESTAMPTZ DEFAULT NOW(),
        updated_at    TIMESTAMPTZ DEFAULT NOW()
    );

```

-- Relación doctor-paciente

```

CREATE TABLE accounts_doctor_patient (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    doctor_id     UUID REFERENCES accounts_doctor_profile(id) ON DELETE CASCADE,
    patient_id    UUID REFERENCES accounts_patient_profile(id) ON DELETE CASCADE,
    assigned_at   TIMESTAMPTZ DEFAULT NOW(),
    is_active     BOOLEAN DEFAULT TRUE,
    notes         TEXT,
    UNIQUE(doctor_id, patient_id)
);

```

4.2 — App: medications

-- Medicamentos base (catálogo)

```

CREATE TABLE medications_medication (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    patient_id    UUID REFERENCES accounts_patient_profile(id) ON DELETE CASCADE,
    name          VARCHAR(200) NOT NULL,
    active_compound VARCHAR(200),
    dosage        VARCHAR(100),
    unit          VARCHAR(50),          -- mg, ml, comprimido
    notes         TEXT,
    is_active     BOOLEAN DEFAULT TRUE,
    created_at    TIMESTAMPTZ DEFAULT NOW(),
    updated_at    TIMESTAMPTZ DEFAULT NOW()
);

```

-- Patrones de medicación

```

CREATE TABLE medications_pattern (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    medication_id UUID REFERENCES medications_medication(id) ON DELETE
    CASCADE,
    pattern_type  VARCHAR(20) NOT NULL,          --
    DAILY|WEEKLY|CUSTOM|SPECIFIC_DAYS
    is_active     BOOLEAN DEFAULT TRUE,
    repeat_every_days INTEGER,
    repeat_for_days INTEGER,
    specific_days_of_week INTEGER[],            -- [1,2,3,4,5] = Lun-Vie
    pause_start_date DATE,
    pause_end_date DATE,
    pause_duration_days INTEGER,
    pause_interval_days INTEGER,
    created_at    TIMESTAMPTZ DEFAULT NOW(),
    updated_at    TIMESTAMPTZ DEFAULT NOW()
);

```

```
);
```

```
-- Horarios de medicación
```

```
CREATE TABLE medications_schedule (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  medication_id  UUID REFERENCES medications_medication(id) ON DELETE  
CASCADE,  
  pattern_id    UUID REFERENCES medications_pattern(id) ON DELETE SET NULL,  
  time         TIME NOT NULL,  
  is_relative_to_meal BOOLEAN DEFAULT FALSE,  
  meal_type    VARCHAR(20),          -- BREAKFAST|LUNCH|DINNER|SNACK  
  offset_minutes INTEGER DEFAULT 0,  
  reminder_minutes_before INTEGER DEFAULT 15,  
  is_active    BOOLEAN DEFAULT TRUE,  
  created_at   TIMESTAMPTZ DEFAULT NOW()  
);
```

```
-- Horarios de comidas
```

```
CREATE TABLE medications_meal_schedule (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  patient_id  UUID REFERENCES accounts_patient_profile(id) ON DELETE CASCADE,  
  meal_name   VARCHAR(100),  
  meal_type   VARCHAR(20) NOT NULL,  
  time       TIME NOT NULL,  
  is_active   BOOLEAN DEFAULT TRUE,  
  is_default  BOOLEAN DEFAULT FALSE,  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
);
```

```
-- Historial de tomas (tabla de alta frecuencia → ideal TimescaleDB)
```

```
CREATE TABLE medications_history (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  patient_id  UUID REFERENCES accounts_patient_profile(id) ON DELETE CASCADE,  
  medication_id  UUID REFERENCES medications_medication(id) ON DELETE  
CASCADE,  
  schedule_id  UUID REFERENCES medications_schedule(id) ON DELETE SET NULL,  
  medication_name VARCHAR(200) NOT NULL,      -- desnormalizado para historial  
  scheduled_at  TIMESTAMPTZ NOT NULL,  
  actual_taken_at TIMESTAMPTZ,  
  reminder_sent_at TIMESTAMPTZ,  
  status       VARCHAR(20) NOT NULL,          --  
TAKEN|SKIPPED|POSTPONED|PENDING  
  dosage_taken VARCHAR(100),  
  notes        TEXT,  
  side_effects TEXT,  
  created_at   TIMESTAMPTZ DEFAULT NOW()  
);
```

```
-- Índice para consultas de adherencia
CREATE INDEX idx_medications_history_patient_date
  ON medications_history(patient_id, scheduled_at DESC);
```

4.3 — App: vitals (con TimescaleDB para series de tiempo)

```
-- Registros de signos vitales
```

```
-- Esta tabla se convierte en hypertable con TimescaleDB
```

```
CREATE TABLE vitals_record (
  id          UUID DEFAULT gen_random_uuid(),
  patient_id  UUID NOT NULL REFERENCES accounts_patient_profile(id) ON DELETE
  CASCADE,
  recorded_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  recorded_by UUID REFERENCES accounts_user(id), -- NULL = autoregistro
```

```
-- Signos vitales
```

```
weight_kg      DECIMAL(5,2),
height_cm      DECIMAL(5,2),
heart_rate     INTEGER,      -- bpm
respiratory_rate INTEGER,    -- rpm
systolic_bp    INTEGER,      -- mmHg
diastolic_bp   INTEGER,      -- mmHg
oxygen_saturation DECIMAL(5,2), -- %
temperature_celsius DECIMAL(4,2), -- °C
```

```
-- Laboratorio
```

```
glucose_mgdl    DECIMAL(6,2),
triglycerides_mgdl DECIMAL(6,2),
urea_mgdl       DECIMAL(6,2),
creatinine_mgdl DECIMAL(6,2),
hemoglobin_gdl  DECIMAL(5,2),
```

```
notes          TEXT,
document_id     VARCHAR(255), -- referencia a Storage
```

```
PRIMARY KEY (id, recorded_at)
```

```
);
```

```
-- TimescaleDB hypertable (ejecutar post-creación)
```

```
-- SELECT create_hypertable('vitals_record', 'recorded_at');
```

```
-- Valores de referencia por tipo de vital y demografía
```

```
CREATE TABLE vitals_reference_range (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vital_type  VARCHAR(50) NOT NULL,
  sex        VARCHAR(10),
  age_min     INTEGER,
  age_max     INTEGER,
  normal_min  DECIMAL(10,4),
```

```

    normal_max    DECIMAL(10,4),
    unit          VARCHAR(20),
    description    TEXT
);

```

4.4 — App: prescriptions

-- Recetas digitales

```

CREATE TABLE prescriptions_prescription (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    patient_id  UUID REFERENCES accounts_patient_profile(id) ON DELETE CASCADE,
    doctor_id   UUID REFERENCES accounts_doctor_profile(id) ON DELETE SET NULL,
    prescription_date DATE NOT NULL DEFAULT CURRENT_DATE,
    diagnosis   TEXT,
    notes       TEXT,
    file_url    VARCHAR(500),          -- PDF en Storage
    is_active   BOOLEAN DEFAULT TRUE,
    created_at  TIMESTAMPTZ DEFAULT NOW()
);

```

-- Líneas de receta (medicamentos prescritos)

```

CREATE TABLE prescriptions_line (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    prescription_id UUID REFERENCES prescriptions_prescription(id) ON DELETE
CASCADE,
    medication_name VARCHAR(200) NOT NULL,
    dosage          VARCHAR(100),
    frequency        VARCHAR(100),
    duration         VARCHAR(100),
    instructions     TEXT
);

```

-- Resultados de laboratorio

```

CREATE TABLE prescriptions_lab_result (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    patient_id  UUID REFERENCES accounts_patient_profile(id) ON DELETE CASCADE,
    doctor_id   UUID REFERENCES accounts_doctor_profile(id) ON DELETE SET NULL,
    lab_date    DATE NOT NULL DEFAULT CURRENT_DATE,
    lab_name    VARCHAR(200),
    results     JSONB,                -- resultados flexibles
    file_url    VARCHAR(500),
    notes       TEXT,
    created_at  TIMESTAMPTZ DEFAULT NOW()
);

```

4.5 — App: surveys

-- Encuestas

```

CREATE TABLE surveys_survey (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```



```

    title      VARCHAR(300) NOT NULL,
    description TEXT,
    doctor_id   UUID REFERENCES accounts_doctor_profile(id) ON DELETE SET NULL,
    is_active   BOOLEAN DEFAULT TRUE,
    created_at  TIMESTAMPTZ DEFAULT NOW()
);

```

-- Preguntas

```

CREATE TABLE surveys_question (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    survey_id   UUID REFERENCES surveys_survey(id) ON DELETE CASCADE,
    text        TEXT NOT NULL,
    question_type VARCHAR(20) NOT NULL,      -- RATING|MULTIPLE|TEXT|YESNO
    options     JSONB,                       -- para MULTIPLE
    order_num   INTEGER DEFAULT 0
);

```

-- Códigos de acceso a encuestas

```

CREATE TABLE surveys_access_code (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    survey_id   UUID REFERENCES surveys_survey(id) ON DELETE CASCADE,
    code        VARCHAR(20) UNIQUE NOT NULL,
    max_uses    INTEGER,
    uses_count  INTEGER DEFAULT 0,
    expires_at  TIMESTAMPTZ,
    is_active   BOOLEAN DEFAULT TRUE,
    created_at  TIMESTAMPTZ DEFAULT NOW()
);

```

-- Respuestas

```

CREATE TABLE surveys_response (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    survey_id   UUID REFERENCES surveys_survey(id) ON DELETE CASCADE,
    patient_id  UUID REFERENCES accounts_patient_profile(id) ON DELETE SET NULL,
    access_code_id UUID REFERENCES surveys_access_code(id) ON DELETE SET
NULL,
    answers     JSONB NOT NULL,              -- {question_id: answer}
    submitted_at TIMESTAMPTZ DEFAULT NOW()
);

```

4.6 — App: audit_logs (sistema propio de logs)

-- Log de auditoría y eventos

```

CREATE TABLE audit_logs_event (
    id          UUID DEFAULT gen_random_uuid(),
    timestamp   TIMESTAMPTZ NOT NULL DEFAULT NOW(),

```

-- Quién

```

    user_id     UUID REFERENCES accounts_user(id) ON DELETE SET NULL,

```

```

session_id    VARCHAR(255),
ip_address    INET,
user_agent    TEXT,

-- Qué
event_type    VARCHAR(50) NOT NULL,      -- API_CALL|AUTH|ERROR|BUSINESS
severity      VARCHAR(20) NOT NULL,      -- INFO|WARNING|ERROR|CRITICAL
category      VARCHAR(50),               -- AUTH|MEDICATION|VITALS|etc.
action        VARCHAR(100),              -- login|take_medication|record_vital

-- Dónde
endpoint      VARCHAR(500),
method        VARCHAR(10),
status_code   INTEGER,
response_ms   INTEGER,

-- Contexto
patient_id    UUID REFERENCES accounts_patient_profile(id) ON DELETE SET NULL,
resource_type VARCHAR(50),
resource_id   UUID,

-- Payload
request_data  JSONB,
response_data JSONB,
error_data    JSONB,
sentry_event_id VARCHAR(255),            -- link a Sentry

-- Metadata
app_version   VARCHAR(20),
platform      VARCHAR(20),              -- ANDROID|WEB|API

PRIMARY KEY (id, timestamp)
);

```

```

-- Hypertable para logs también
-- SELECT create_hypertable('audit_logs_event', 'timestamp');

```

```

CREATE INDEX idx_audit_logs_user ON audit_logs_event(user_id, timestamp DESC);
CREATE INDEX idx_audit_logs_severity ON audit_logs_event(severity, timestamp DESC);
CREATE INDEX idx_audit_logs_patient ON audit_logs_event(patient_id, timestamp DESC);

```

4.7 — App: analytics (vistas materializadas para el dashboard)

```

-- Vista materializada: adherencia diaria por paciente
CREATE MATERIALIZED VIEW analytics_medication_adherence AS
SELECT
  mh.patient_id,
  DATE(mh.scheduled_at) as date,
  COUNT(*) as total_doses,

```

```

COUNT(*) FILTER (WHERE mh.status = 'TAKEN') as taken_doses,
ROUND(
    COUNT(*) FILTER (WHERE mh.status = 'TAKEN')::NUMERIC /
    NULLIF(COUNT(*), 0) * 100, 2
) as adherence_pct
FROM medications_history mh
GROUP BY mh.patient_id, DATE(mh.scheduled_at);

```

```

-- Vista materializada: últimos vitales por paciente
CREATE MATERIALIZED VIEW analytics_latest_vitals AS
SELECT DISTINCT ON (patient_id)
    patient_id,
    recorded_at,
    weight_kg, heart_rate, systolic_bp, diastolic_bp,
    oxygen_saturation, temperature_celsius, glucose_mgdl
FROM vitals_record
ORDER BY patient_id, recorded_at DESC;

```

```

-- Índice para refresh rápido
CREATE UNIQUE INDEX ON analytics_medication_adherence(patient_id, date);
CREATE UNIQUE INDEX ON analytics_latest_vitals(patient_id);

```

5. DJANGO APPS — MODELOS PRINCIPALES

```

apps/medications/models.py (ejemplo representativo)
from django.db import models
import uuid

```

```

class Medication(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4)
    patient = models.ForeignKey('patients.PatientProfile', on_delete=models.CASCADE)
    name = models.CharField(max_length=200)
    active_compound = models.CharField(max_length=200, blank=True)
    dosage = models.CharField(max_length=100, blank=True)
    unit = models.CharField(max_length=50, blank=True)
    notes = models.TextField(blank=True)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

```

```

class Meta:
    db_table = 'medications_medication'

```

```

class MedicationHistory(models.Model):
    class Status(models.TextChoices):
        TAKEN = 'TAKEN'
        SKIPPED = 'SKIPPED'
        POSTPONED = 'POSTPONED'
        PENDING = 'PENDING'

```

```

id = models.UUIDField(primary_key=True, default=uuid.uuid4)
patient = models.ForeignKey('patients.PatientProfile', on_delete=models.CASCADE)
medication = models.ForeignKey(Medication, on_delete=models.CASCADE)
medication_name = models.CharField(max_length=200)
scheduled_at = models.DateTimeField()
actual_taken_at = models.DateTimeField(null=True, blank=True)
status = models.CharField(max_length=20, choices=Status.choices)
notes = models.TextField(blank=True)
side_effects = models.TextField(blank=True)
created_at = models.DateTimeField(auto_now_add=True)

class Meta:
    db_table = 'medications_history'
    indexes = [
        models.Index(fields=['patient', '-scheduled_at'])
    ]

```

6. SEGURIDAD — CLERK + SENTRY

6.1 Clerk (Autenticación)

Flujo de autenticación:

Android App → Clerk SDK → JWT Token

JWT Token → Django Backend → Clerk JWKS verification

Django → extrae clerk_id → busca/crea User local

Roles soportados:

- PATIENT → acceso solo a sus propios datos
- DOCTOR → acceso a sus pacientes asignados
- ADMIN → acceso total + gestión de usuarios

middleware/clerk_auth.py

import jwt

from clerk_backend_api import Clerk

class ClerkAuthMiddleware:

"""Verifica JWT de Clerk y adjunta user al request."""

def authenticate(self, request):

token = self._extract_bearer(request)

payload = Clerk().verify_token(token)

clerk_id = payload['sub']

user = User.objects.get(clerk_id=clerk_id)

return user

permissions.py

class IsDoctor(BasePermission):

def has_permission(self, request, view):

```
return request.user.role == 'DOCTOR'
```

```
class IsPatientOwner(BasePermission):  
    def has_object_permission(self, request, view, obj):  
        return obj.patient.user == request.user
```

6.2 Sentry — Integración Django

```
# config/settings/base.py  
import sentry_sdk  
from sentry_sdk.integrations.django import DjangoIntegration  
from sentry_sdk.integrations.celery import CeleryIntegration  
  
sentry_sdk.init(  
    dsn=env('SENTRY_DSN'),  
    integrations=[DjangoIntegration(), CeleryIntegration()],  
    traces_sample_rate=0.2,    # 20% de requests para performance  
    profiles_sample_rate=0.1,  
    send_default_pii=False,    # HIPAA: no enviar PII a Sentry  
    before_send=sanitize_medical_data, # filtrar datos médicos  
    environment=env('ENVIRONMENT'),  
)
```

6.3 Custom Log Dashboard (propio en Django)

```
# apps/audit_logs/middleware.py  
class AuditLogMiddleware:  
    """Captura cada request y lo persiste en audit_logs_event."""  
  
    def process_response(self, request, response):  
        AuditLogEvent.objects.create(  
            user_id=getattr(request.user, 'id', None),  
            event_type='API_CALL',  
            severity=self._severity_from_status(response.status_code),  
            endpoint=request.path,  
            method=request.method,  
            status_code=response.status_code,  
            response_ms=self._elapsed_ms(request),  
            ip_address=self._get_client_ip(request),  
            platform=request.headers.get('X-Platform', 'API'),  
            app_version=request.headers.get('X-App-Version'),  
        )  
        return response
```

7. DASHBOARD DEL DOCTOR

Secciones del dashboard

Doctor Dashboard

└─ 1. Overview / Home

- Total pacientes activos
- Alertas del día (vitales fuera de rango)
- Pacientes con baja adherencia (<70%)
- Próximas consultas
- 2. Mis Pacientes (listado)
 - Búsqueda y filtros
 - Estado de adherencia por paciente
 - Último vital registrado
- 3. Ficha del Paciente (drill-down)
 - Datos personales y demográficos
 - Medicamentos activos
 - Histórico de adherencia (gráfica)
 - Signos vitales (series de tiempo)
 - Presión arterial, FC, SpO2, Temperatura
 - Laboratorio: glucosa, triglicéridos, etc.
 - Recetas emitidas
 - Resultados de lab
 - Encuestas respondidas
- 4. Analytics Globales
 - Adherencia promedio de cohorte
 - Distribución de adherencia por medicamento
 - Tendencias de vitales por grupo
 - Tasa de respuesta a encuestas
- 5. Alertas & Notificaciones
 - Vitales fuera de rango (tiempo real via WebSocket)
 - Pacientes sin registros recientes
 - Historial de alertas
- 6. Logs & Auditoría (custom Sentry-like panel)
 - Timeline de eventos por paciente
 - Errores de la app (sincronizado con Sentry)
 - Actividad de usuarios
 - Filtros: severity, tipo, fecha, paciente
 - Exportar logs CSV
- 7. Configuración
 - Perfil del doctor
 - Gestión de encuestas propias
 - Rangos de referencia personalizados

API endpoints para el dashboard

GET /api/v1/dashboard/overview/ → métricas generales

GET /api/v1/dashboard/patients/ → lista pacientes

GET /api/v1/dashboard/patients/{id}/ → ficha completa

```

GET /api/v1/dashboard/patients/{id}/vitals/?from=&to=&type=
GET /api/v1/dashboard/patients/{id}/adherence/?period=30d
GET /api/v1/dashboard/alerts/ → alertas activas
GET /api/v1/dashboard/analytics/ → métricas globales
GET /api/v1/dashboard/logs/?severity=&from=&to=&patient=
WS /ws/dashboard/alerts/ → alertas en tiempo real

```

8. CELERY — TAREAS EN BACKGROUND

```
# apps/notifications/tasks.py
```

```
@shared_task
```

```
def check_medication_adherence():
```

```
    """Ejecuta cada hora. Detecta pacientes con baja adherencia."""
```

```
    ...
```

```
@shared_task
```

```
def send_medication_reminder(schedule_id, patient_id):
```

```
    """Dispara notificación push al paciente."""
```

```
    ...
```

```
@shared_task
```

```
def refresh_analytics_views():
```

```
    """Actualiza vistas materializadas cada 15 min."""
```

```
    ...
```

```
@shared_task
```

```
def alert_doctor_vital_out_of_range(vital_record_id):
```

```
    """Notifica al doctor si un vital está fuera de rango."""
```

```
    ...
```

9. PLAN DE MIGRACIÓN (Firestore → PostgreSQL)

Fase	Acción	Duración estimada
1	Setup Django + Postgres + Clerk + Sentry	Sprint 1
2	Modelos, migraciones, endpoints Auth	Sprint 1
3	API Medicamentos + Vitals + sync Android	Sprint 2
4	Doctor Dashboard (Next.js o Django Templates)	Sprint 3
5	Migración de datos Firestore → Postgres	Sprint 3
6	Custom Log Dashboard	Sprint 4
7	Notificaciones (Celery + FCM/Expo Push)	Sprint 4
8	Analytics, vistas materializadas, alertas	Sprint 5
9	QA, hardening seguridad, go-live	Sprint 6

10. STACK TECNOLÓGICO FINAL RECOMENDADO

Capa	Tecnología	Justificación
Backend	Python 3.12 + Django 5.x + DRF	Robusto, ecosistema médico, ORM potente
DB principal	PostgreSQL 16 + TimescaleDB	Integridad relacional + series de tiempo vitales

Cache/Queue	Redis 7	Sesiones, cache de vistas, cola Celery
Workers	Celery 5	Notificaciones, analytics, tareas scheduled
Auth	Clerk	JWT, MFA, roles, Google/Apple login
Logging	Sentry SDK + audit_logs custom	Crashes + historial médico auditable
Android	Kotlin/Compose (actual) + Retrofit	Migrar llamadas a Django API
Dashboard	Django Admin customizado (fase 1) → Next.js (fase 2)	Rápido al inicio, escalable
Storage	Cloudflare R2 o AWS S3	PDFs, fotos, recetas
Infra	Docker + Nginx + Gunicorn	Contenedorizado, portable
CI/CD	GitHub Actions	Builds, tests, deploy automático

MAILYTCUIDA — PRODUCTO, NEGOCIO Y ARQUITECTURA COMPLETA

MODELO DE NEGOCIO — MEMBRESÍAS

Fundamentado en casos de éxito reales

MyFitnessPal → gamificación + tracking diario = retención 65%

Noom → coaching IA + humano = conversión freemium 30%

Whoop → análisis continuo de métricas = ARPU \$18/mes

ZocDoc → marketplace especialistas = comisión por booking

Hinge Health → fisioterapia digital = \$1,500 MXN/mes promedio

PLANES DE MEMBRESÍA

FREE	SILVER	GOLD	PLATINUM
\$0/mes	\$99/mes	\$249/mes	\$499/mes
• Vitales	• +Gráficas	• +IA análisis	• +Nutriólogo
• Meds básico	• +PDF export	• +Alertas doc	• +Fisioterapeuta
• Calendario	• +Notif. intel	• +1 consulta/	• +Chat directo
• Recetas	• +2x puntos	• mes gratis	• +2 consultas/mes
• 100 pts	• +10% tienda	• +20% dcto	• +30% dcto
	• +3x puntos	• +5x puntos	
		• +Cupones VIP	

FUENTES DE INGRESO (Modelo Multi-revenue)

Fuente	Modelo	Estimado
Suscripciones pacientes	B2C recurrente	Base principal
Comisión por booking especialista	15% por consulta agendada	Escalable
Venta productos clínica	Margen 20-40% sobre PVP	Inmediato
Clinic Partners (B2B)	Fee mensual por clínica usando plataforma	Escalable
Cupones empresas externas	CPL/CPA con Cinopolis, farmacias, etc.	Pasivo
Planes familiares	+\$149/miembro adicional hasta 4	Expansión

MÓDULOS COMPLETOS DEL SISTEMA

Módulos actuales (ya existen en app Android)

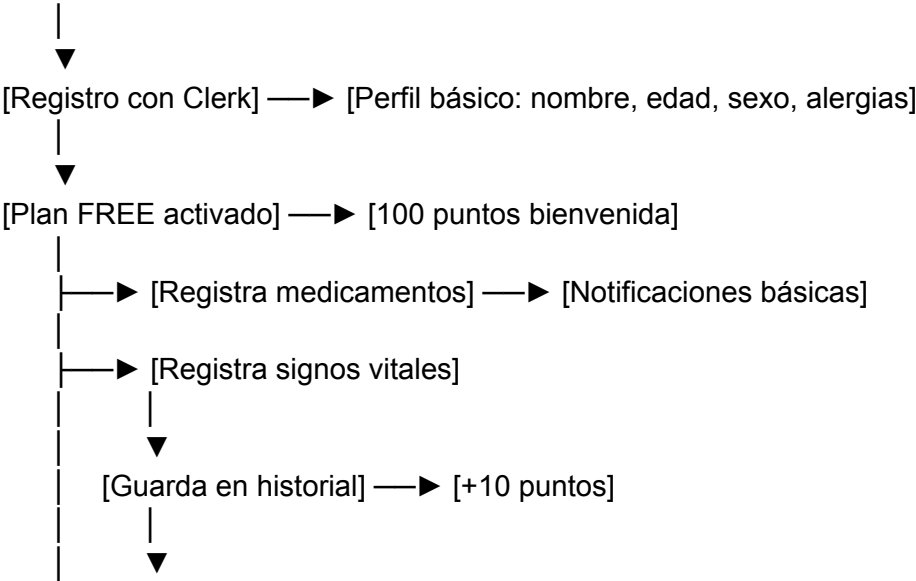
Módulos nuevos a construir

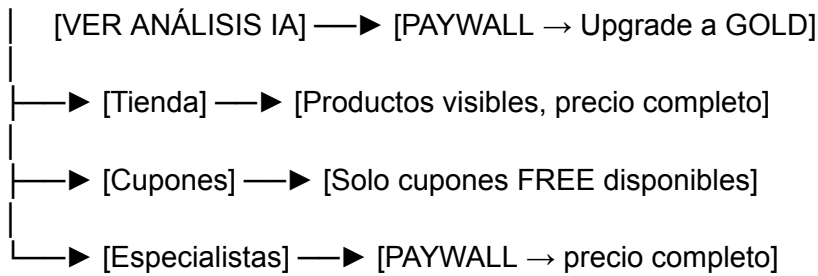
MÓDULOS NUEVOS — 12 adicionales		
1. SUBSCRIPTIONS	Stripe + planes + upgrades	
2. AI HEALTH ENGINE	Claude/OpenAI análisis vitales	
3. GAMIFICATION	Puntos, racha, niveles, logros	
4. COUPON ENGINE	Clínica + empresas externas	
5. STORE	Productos clínica + carrito	
6. SPECIALISTS	Marketplace nutriólogos/fisios	
7. TELEMEDICINE	Video llamada con especialista	
8. NUTRITION	Plan de comidas + seguimiento	
9. WELLNESS	Sueño, agua, pasos, bienestar	
10. FAMILY PLAN	Perfiles familiares vinculados	
11. CHALLENGES	Retos de salud comunitarios	
12. PARTNER PORTAL	Portal para empresas externas	

DIAGRAMA DE FLUJO COMPLETO — CASOS DE USO

ACTOR: PACIENTE (FREE)

[Descarga app]





ACTOR: PACIENTE (GOLD)

[Registra signos vitales]

—> [+15 puntos si incluye foto de evidencia en peso]

▼
[Motor IA analiza automáticamente]

—> [Todos los valores OK]

▼
[Notificación paciente: "Todo bien, sigue así"]
[Retroalimentación OMS: "Tu presión es óptima para..."]

—> [Valor fuera de rango]

▼
[Alerta al doctor asignado] —> [Push notification doctor]
[Recomendación IA con fuente OMS]
[Paciente: "Consulta a tu médico"]

[Solicita consulta especialista]

▼
[Marketplace especialistas]

—> [1 consulta gratuita disponible este mes?]

▼
Sí

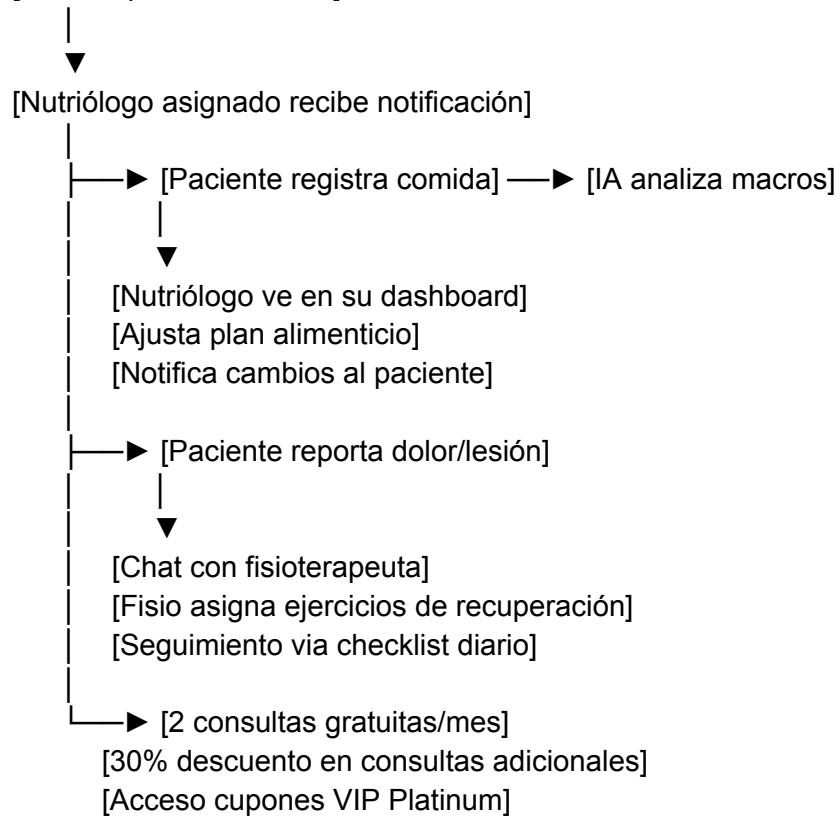
[Agenda gratuita] —> [Video llamada in-app]

—> NO: [Precio con 20% descuento GOLD]

▼
[Pago via Stripe]
[Booking confirmado]
[Video llamada schedulada]

ACTOR: PACIENTE (PLATINUM)

[Perfil deportivo activado]



SISTEMA DE PUNTOS Y GAMIFICACIÓN

ACUMULACIÓN DE PUNTOS:

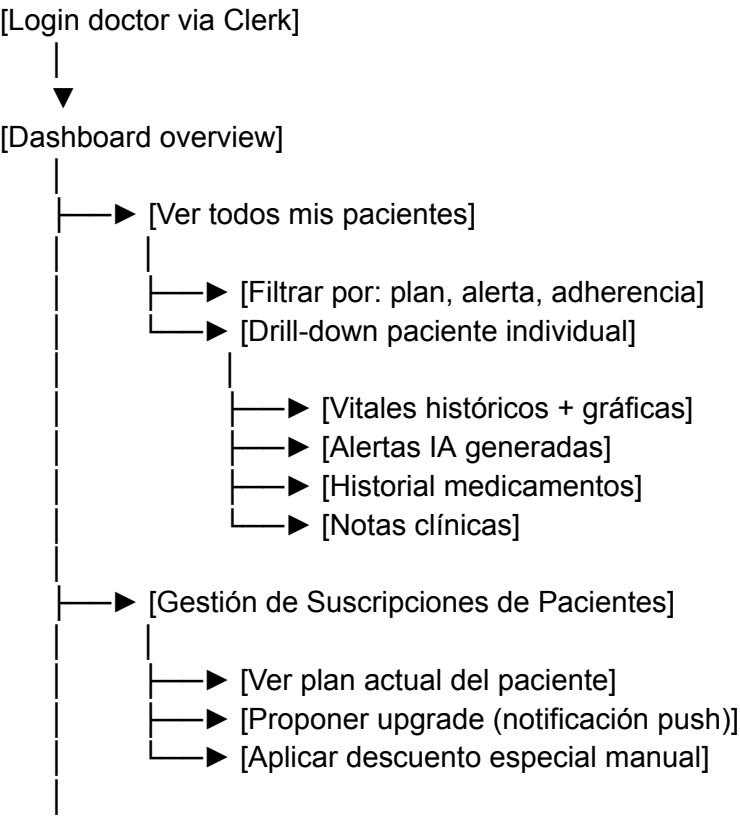
Acción	Puntos base	
Registrar vitales	+10	
Tomar medicamento a tiempo	+5	

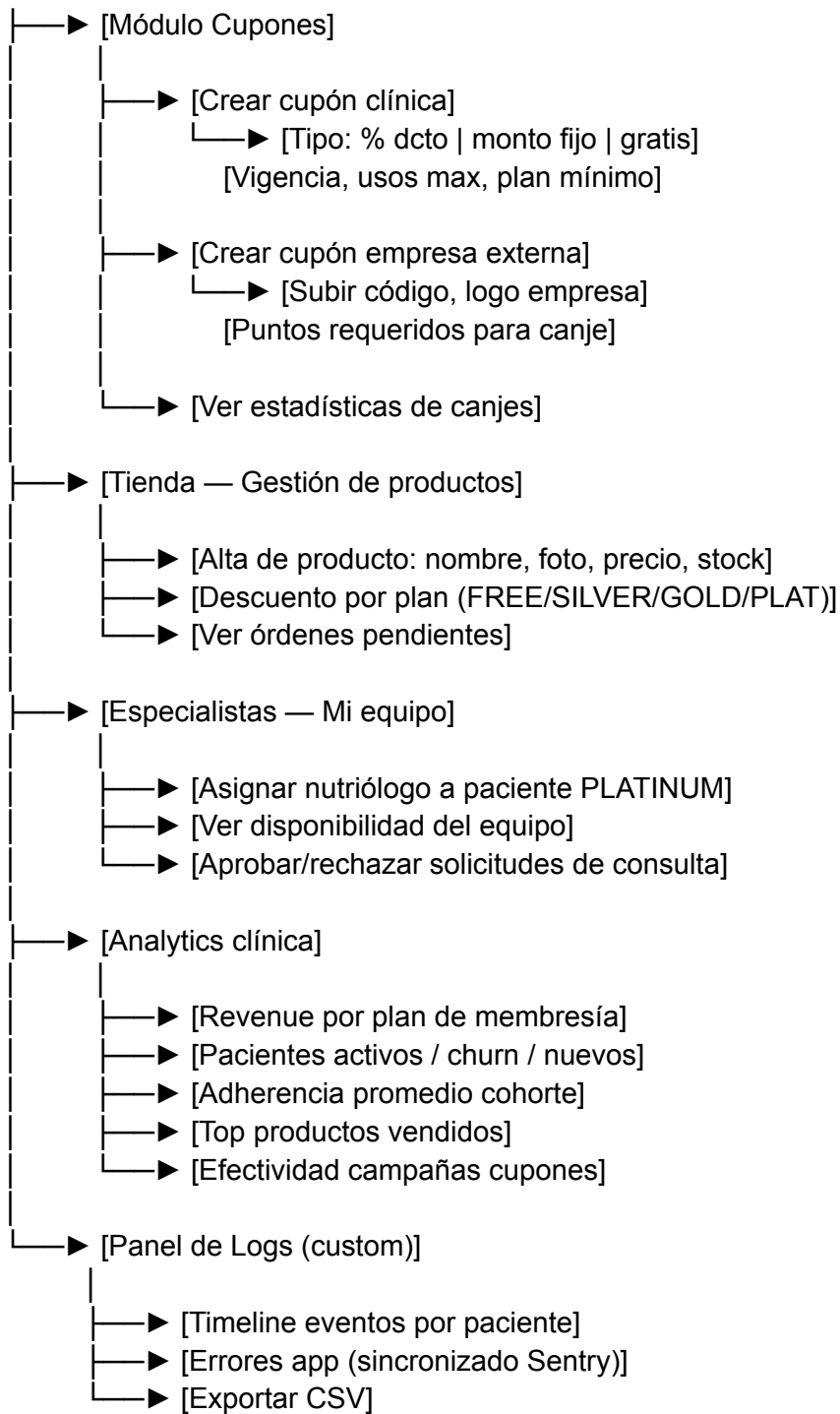
Foto de evidencia (peso)	+15 extra	
Peso mejorado vs anterior	+20	
Racha 7 días consecutivos	+50 bonus	
Racha 30 días	+200 bonus	
Completar perfil 100%	+100 (1 vez)	
Referir a un amigo	+150	
Completar encuesta	+25	
Responder reto semanal	+75	

MULTIPLICADORES POR PLAN:
FREE: 1x | SILVER: 2x | GOLD: 3x | PLATINUM: 5x

CANJE DE PUNTOS:
500 pts → Cupón 10% en consulta
1,000 pts → Cupón en tienda clínica (\$50 pesos)
2,500 pts → Consulta gratuita básica
5,000 pts → Cupón Cinépolis / socio externo
10,000 pts → Upgrade 1 mes a plan superior

ACTOR: DOCTOR (Dashboard)



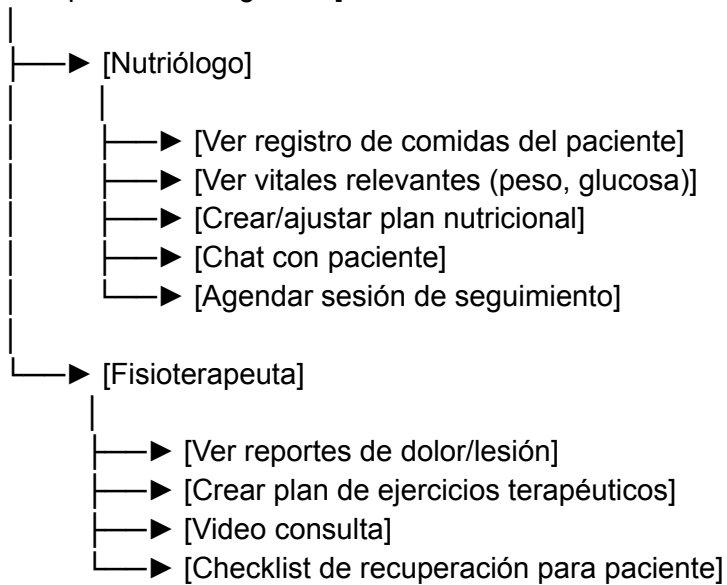


ACTOR: ESPECIALISTA (Nutriólogo / Fisioterapeuta)

[Login especialista via Clerk]



[Ver mis pacientes asignados]

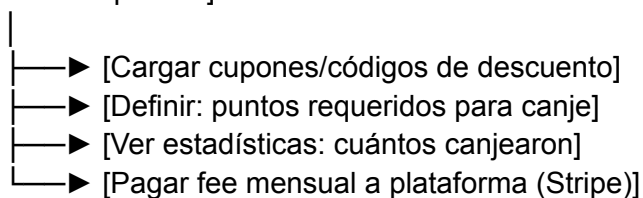


ACTOR: PARTNER EXTERNO (Cinepolis, Farmacia, etc.)

[Login Portal Partner]



[Dashboard partner]



MÓDULOS DE BIENESTAR EXTRA (inspirados en casos de éxito)

MÓDULO	INSPIRACIÓN	FEATURE CLAVE
--------	-------------	---------------

Seguimiento sueño	Whoop / Sleep Cycle	Calidad + horas + puntos
Hidratación	WaterMinder	Vasos de agua + recordatorios
Pasos / Actividad	Google Fit / Strava	Integración wearables
Ciclo menstrual	Flo App	Predicciones + alertas
Retos de salud	Noom / Strava	Retos con comunidad
Diario emocional	Headspace	Check-in mental + IA
Síntomas checker	Ada Health	IA + disclaimer médico
Familia	Apple Family	Hasta 4 perfiles vinculados
Contenido educativo	MedlinePlus	Artículos por condición
Historial médico	PicnicHealth	Almacenamiento docs, CDMX

ESQUEMA DE BASE DE DATOS ADICIONAL

Módulo Subscriptions

-- Planes disponibles

```
CREATE TABLE subscriptions_plan (  
  id      UUID PRIMARY KEY,  
  name    VARCHAR(20), -- FREE|SILVER|GOLD|PLATINUM  
  price_mxn DECIMAL(8,2),  
  ai_analysis BOOLEAN,  
  points_mult INTEGER,  
  free_consults_per_month INTEGER,  
  specialist_discount_pct INTEGER,  
  store_discount_pct INTEGER  
);
```

-- Suscripción activa del usuario

```
CREATE TABLE subscriptions_subscription (  
  id      UUID PRIMARY KEY,  
  user_id  UUID REFERENCES accounts_user(id),  
  plan_id  UUID REFERENCES subscriptions_plan(id),  
  stripe_sub_id VARCHAR(255) UNIQUE,  
  status   VARCHAR(20), -- ACTIVE|CANCELLED|PAST_DUE  
  current_period_start TIMESTAMPTZ,  
  current_period_end   TIMESTAMPTZ,  
  cancelled_at TIMESTAMPTZ  
);
```

-- Puntos del paciente

```
CREATE TABLE gamification_wallet (  
  id      UUID PRIMARY KEY,  
  patient_id UUID UNIQUE REFERENCES accounts_patient_profile(id),  
  balance  INTEGER DEFAULT 0,  
  total_earned INTEGER DEFAULT 0,  
  total_spent INTEGER DEFAULT 0,  
  updated_at TIMESTAMPTZ  
);
```

-- Transacciones de puntos

```
CREATE TABLE gamification_transaction (  
  id      UUID DEFAULT gen_random_uuid(),  
  timestamp TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  patient_id UUID REFERENCES accounts_patient_profile(id),  
  points    INTEGER, -- positivo = ganó, negativo = gastó  
  action_type VARCHAR(50), --  
  VITAL_RECORDED|MED_TAKEN|COUPON_REDEEMED  
  description TEXT,  
  reference_id UUID,  
  PRIMARY KEY (id, timestamp)  
);
```

-- Cupones

```
CREATE TABLE coupons_coupon (  
  id      UUID PRIMARY KEY,  
  code    VARCHAR(50) UNIQUE NOT NULL,  
  title   VARCHAR(200),  
  description TEXT,  
  coupon_type VARCHAR(20), -- CLINIC|EXTERNAL  
  discount_type VARCHAR(20), -- PERCENT|FIXED|FREE_CONSULT  
  discount_value DECIMAL(8,2),  
  points_required INTEGER DEFAULT 0,  
  min_plan VARCHAR(20), -- NULL=todos, GOLD, PLATINUM  
  valid_from TIMESTAMPTZ,  
  valid_until TIMESTAMPTZ,  
  max_uses  INTEGER,  
  uses_count INTEGER DEFAULT 0,  
  partner_id UUID,      -- NULL si es de clínica  
  created_by UUID REFERENCES accounts_doctor_profile(id),  
  is_active BOOLEAN DEFAULT TRUE  
);
```

-- Tienda

```
CREATE TABLE store_product (  
  id      UUID PRIMARY KEY,  
  name    VARCHAR(200),  
  description TEXT,  
  photo_url VARCHAR(500),  
  price_mxn DECIMAL(8,2),  
  stock   INTEGER,  
  category VARCHAR(100),  
  discount_free INTEGER DEFAULT 0, -- % por plan  
  discount_silver INTEGER DEFAULT 5,  
  discount_gold INTEGER DEFAULT 10,  
  discount_platinum INTEGER DEFAULT 20,  
  is_active BOOLEAN DEFAULT TRUE  
);
```

-- Órdenes de tienda

```
CREATE TABLE store_order (  
  id      UUID PRIMARY KEY,  
  patient_id UUID REFERENCES accounts_patient_profile(id),  
  stripe_payment_id VARCHAR(255),  
  status   VARCHAR(20), -- PENDING|PAID|SHIPPED|DELIVERED  
  total_mxn DECIMAL(8,2),  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

ARQUITECTURA BACKEND FINAL — TODOS LOS APPS DJANGO

apps/

— accounts/	# Users, roles, Clerk sync
— patients/	# Patient profiles, doctor-patient relations
— medications/	# Meds, patterns, schedules, history
— vitals/	# Signs, lab results, TimescaleDB
— prescriptions/	# Prescriptions, lab docs
— calendar_events/	# Events, reminders
— surveys/	# Surveys, codes, responses
— subscriptions/	# Plans, Stripe billing
— ai_engine/	# Claude/OpenAI analysis, OMS references
— gamification/	# Points wallet, transactions, levels
— coupons/	# Coupon engine, redemptions
— store/	# Products, cart, orders
— specialists/	# Nutritionists, physios, booking
— telemedicine/	# Video sessions (Twilio/Daily.co)
— nutrition/	# Meal plans, food tracking
— wellness/	# Sleep, water, steps, mood
— notifications/	# Push, email, SMS center
— analytics/	# Metrics, KPIs, materialized views
— audit_logs/	# Custom log panel + Sentry sync
— dashboard/	# Doctor + specialist dashboard views
— partner_portal/	# External business portal

PREGUNTAS QUE NECESITO QUE RESPONDAS

Para arrancar el código tengo que saber estos 6 puntos:

1. Procesador de pagos

¿Stripe internacional o también Conekta (para México)? ¿O ambos?

2. Mercado objetivo

¿México exclusivamente al inicio o Latinoamérica desde el arranque? Afecta monedas, idiomas y compliance.

3. Modelo de clínica

¿Esto es una clínica propia que usa la app, o es una plataforma que venden a múltiples clínicas (multi-tenant)?

4. IA para análisis de vitales

¿Usamos Claude API (Anthropic) u OpenAI? ¿O ambas con fallback?

5. Video consultas

¿Integramos Twilio/Daily.co desde el inicio o las consultas son solo agendamiento + link externo (Zoom/Meet) en fase 1?

1 por ahora lo dejaremos con stripe Internacional y mexico

2 el mercado objetivo sera Mexico pues es donde estamos, mas no esta peleado que se extienda a latinoamerica pues es la intencion

3 esto es CAMSA un corporativo medico, cada que haga alianza con otros medicos o especialistas los sumara a la app para que los pacientes puedan conectar con especialistas con nosotros como intermediario, la intencion es que nosotros eduquemos a los pacientes y asi los pacientes acudan a especialistas y nosotros llevarnos una comision o que sea gratis, eso ya se arreglara aparte pero incluye la logica que el doctor pueda agregar a especialistas, clinicas o incluso laboratorios

4 para temas de costo y que sea mas barato usaremos OpenAI, a menos que sea un platino usaremos Claude

5 usaremos un link externo de meet o zoom, no gastaremos mucho en este modulo

6 <https://github.com/MrFtyoQr/MailyT-CuidaLEGACY.git> este es el repo donde montaremos backend, mobile android y iOS

6. Repo nuevo

¿Cuál es el nombre/URL del nuevo repositorio de GitHub para configurar los remotes en el Markdown?